# TASK-006.1 - Colisão seletiva do Passo Relâmpago

Data: 18/10/2026. Implementação e testes automatizados preparados para revisão. Pendências de aceite: teste reservado ao usuário, conferência jogando na janela e revisão técnica. TASK-007 não iniciada.

## Proposta e decisões

Inspecionados LightningStepService, MovementService, CollisionService/Space, SpatialBodyIndex, DebugWorld, DebugSnapshot, DebugPanel e testes relacionados antes das alterações. Estado inicial: mvn clean test -q, código de saída 0, 147 testes, zero falhas/erros/ignorados nos relatórios Surefire.

A proposta apresentada foi acrescentar categorias e políticas à física existente, registrar os vértices X/Y efetivamente percorridos e corrigir a saída pela mesma geometria. Sem dependências, exclusões ou renomeações. O serviço de habilidade continua responsável pelo relógio e pelo consumo; CollisionService continua responsável por toda a geometria. A simulação mantém uma única thread proprietária.

CollisionCategory explicita ACTOR, SOFT_OBSTACLE, SOLID_OBSTACLE e WORLD_BOUNDARY. CollisionPolicy disponibiliza SOLID e PHASE_THROUGH_ACTORS_AND_SOFT. Ambas bloqueiam sólidos e limites. Obstáculos retangulares aceitam apenas categorias de obstáculo; corpos de entidades são ACTOR. As bordas continuam implícitas nas dimensões de Space, sem construir quatro paredes artificiais.

O construtor antigo Obstacle(x,y,width,height) continua sólido. A política é aplicada na consulta e varredura, não na montagem do índice espacial. Space e SpatialBodyIndex preservam seus contratos e composição; não foi necessário modificar o índice.

## Geometria e saída segura

isFree e resolveContinuous possuem overloads com CollisionPolicy. Os overloads antigos usam SOLID. MovementService.moveContinuous também recebe política opcional; os caminhos do WASD não mudaram. NPCs sem componente de vida bloqueiam normalmente; corpos com vida morta continuam sendo ignorados. Isso permite usar o Npc de diálogo real sem torná-lo combatente.

axisIntervals é a fonte comum dos intervalos analíticos de contato com retângulos arredondados e círculos para varredura e recuperação. Não foi criado outro algoritmo de colisão para o dash. Tangências continuam permitidas; contatos são validados na geometria existente e arredondamentos internos recuam até oito ULPs, como na varredura anterior.

O dash guarda origem, fim do eixo X e fim do eixo Y a cada avanço. Ao terminar, recoverAlongPath consulta as posições atuais de todos os corpos pela política SOLID. Se o destino estiver ocupado, busca o primeiro ponto livre em ordem reversa dessa polilinha. A distância usada para escolher a saída é a distância ao longo da rota, não uma aproximação por corda diagonal. Fronteiras analíticas permitem encontrar até uma tangência compartilhada entre dois intervalos; não há amostragem por pixel.

Cada trecho reverso é limitado pela mesma varredura seletiva contra sólidos e bordas. Uma parede que passe a bloquear o retorno impede examinar pontos além dela. A posição só é alterada após encontrar uma saída válida, e o índice é atualizado imediatamente após a correção. Não há empurrão, dano ou teleporte direto para a origem através de paredes.

Corpos móveis podem ocupar toda a trajetória, tornando impossível encontrar uma saída que satisfaça as regras. Nesse caso o dash termina em 0,25 s, mantém a posição segura perante paredes/bordas e publica awaitingLanding. WASD e uma nova ativação de dash aguardam; cooldown segue diminuindo e não há novo consumo. A busca é repetida a cada atualização com a geometria atual. A sobreposição com ator ou obstáculo leve permanece transitoriamente até haver saída. O jogador continua vulnerável e pode usar os sistemas de combate existentes.

Essa espera não garante que a IA resolverá o bloqueio: um mob com origem sobreposta também pode ficar impedido de mover. Morte, remoção ou movimentação de um bloqueador, ou reinício da arena, podem liberar o estado. Não foram adicionados reserva da origem, empurrão, invulnerabilidade ou uma nova regra de movimentação para contornar essa impossibilidade.

## Habilidade e interface

O construtor anterior de LightningStepService continua SOLID para compatibilidade e regressão. Um novo construtor recebe CollisionPolicy. DebugWorld escolhe explicitamente PHASE_THROUGH_ACTORS_AND_SOFT. O Passo Relâmpago atravessa jogadores, mobs, NPCs sólidos e obstáculos leves durante o deslocamento.

Preservados: 150 px de orçamento direcional, normalização diagonal, velocidade 600 px/s, duração 0,25 s, custo único 35 e cooldown 4 s. O último tick continua fracionário. Colisão sólida reduz a distância sem compensação posterior. A correção pode recuar o destino ao fim; esse recuo é uma resolução de sobreposição, não uma extensão do tempo ou alcance da habilidade. Morte interrompe o avanço e limpa uma espera de saída.

Step.path publica uma cópia imutável dos trechos avançados e eventualmente recuados no tick. O rastro segue esses trechos, sem desenhar cordas que cortem paredes. DebugSnapshot acrescenta somente lightningStepAwaitingLanding; ActorView e Obstacle visuais mantêm o formato usado pelo replay histórico. O painel lê valores e desenha, sem consultar componentes ou atualizar regras.

O HUD informa a travessia seletiva, as paredes/bordas bloqueantes, custo, cooldown e eventual espera. R cria outro mundo e elimina habilidade, caminho e rastros. A fila de comandos e os bindings de Shift não foram alterados. Q e Shift continuam competindo pelo vigor em ordem FIFO.

Os três obstáculos da arena continuam sólidos e nas posições anteriores. Nenhum arbusto foi acrescentado ao cenário de regressão. Obstáculos leves são exercitados nos testes do domínio; a captura demonstra a passagem por um ator real da arena. A grama da interface continua apenas decorativa.

CombatService, Criostase, efeitos de Freeze, cálculo de dano, alcance/cone/cooldown e animação do ataque básico não foram modificados. Sem novas habilidades, dependências, banco, login ou rede.

## Testes e evidências reais

Comando final na pasta Projeto-K/server: mvn clean test. Resultado: BUILD SUCCESS, compilação release 17, 169 testes, zero falhas, zero erros e zero ignorados; tempo Maven 11,917 s. Resumo por classe em TASK-006-1-testes.txt.

São 147 testes anteriores e 22 novos. Os testes antigos permanecem, incluindo o teste de direção zero escrito pelo usuário. MovementReplayTest e seus três baselines não foram modificados. Combate, Criostase, Freeze, consumo, fila, Shift mantido/toques rápidos e reinício passaram na suíte completa.

CollisionPolicyTest: 6 casos para classificação sólida por padrão, consulta seletiva, paredes finas/espessas, deslizamento por eixo, quatro bordas e movimento normal bloqueado por leves. As verificações de parede novas operam diretamente na física; o teste de parede do serviço dash foi reservado ao usuário.

LightningStepSelectiveCollisionTest: 11 casos para atores das três classes, arbustos, construtor sólido anterior, alcance diagonal/bordas, destinos ocupados, união de bloqueadores, recuo pela curva real X/Y, corpos móveis/índice atualizado, ausência de saída com liberação posterior e corpo morto. Preservados dano/efeitos/vigor dos atravessados e parâmetros do dash.

CollisionRecoveryTest: 4 casos para caminho real em volta de construção sólida, parede surgida no retorno, tangência compartilhada e rejeição de rota inválida sem mutação. SelectiveDashIntegrationTest acrescenta 1 caso usando a arena real: comprova hitboxes sobrepostas durante o dash, término livre, distância de 150 px, mobs sem dano e jogador ainda vulnerável.

TASK-006-1-colisao-seletiva.png foi gerada pelo teste e inspecionada visualmente. Mostra dois quadros da renderização real: passagem por um mob e término livre, com hitboxes, rastro, HP, vigor e recarga. A renderização não altera o snapshot. Não houve sessão humana interativa com JFrame; a captura não substitui essa conferência.

## Sua participação reservada

Não foi escrito o teste devePreservarBloqueioDeParedeNoDashSeletivo, nem criado um método vazio. O arquivo recomendado é server/src/test/java/com/projetok/domain/skill/LightningStepSelectiveCollisionTest.java, no pacote com.projetok.domain.skill. Acrescente seu método nessa classe já existente.

Fixtures reais disponíveis por import estático de LightningStepTestFixtures, no mesmo pacote:

- static Player player(float x, float y): fábrica real, HP 100 e vigor 100.
- static final float RADIUS = 18f.

- static LightningStepService selectiveService(): política PHASE_THROUGH_ACTORS_AND_SOFT.
- static LightningStepService service(): política SOLID, compatibilidade anterior.
- static CollisionService.Space emptySpace(): arena 960 por 640, sem obstáculos.

Construtores e métodos reais para montar o seu cenário:

- new CollisionService.Obstacle(float x, float y, float width, float height): parede sólida por padrão.
- new CollisionService.Space(float width, float height, List<Obstacle> obstacles, List<Body> bodies).
- new CollisionService.Body(GameEntity entity, float radius), caso queira incluir o jogador no espaço.
- new SkillCastCommand(String skillId, float targetX, float targetY), com LightningStepService.ID.
- Status cast(Player player, SkillCastCommand command, float radius, CollisionService.Space space).
- Step update(Player player, float dt, float radius, CollisionService.Space space).
- player.getPosition().x()/y(); service.active(), remaining(), cooldown(), awaitingLanding().
- boolean CollisionService.isFree(GameEntity entity, float x, float y, float radius, Space space): validação normal.

O construtor explícito é LightningStepService(MovementService movement, StaminaService stamina, CollisionService collisions, CollisionPolicy policy). Os três serviços devem compartilhar a mesma instância CollisionService. A fixture selectiveService() já faz essa montagem. update aceita delta zero ou positivo finito e limita o tempo de avanço ao restante; a ativação não altera a posição imediatamente.

Sua implementação deve conferir a parede considerando o raio do jogador. A solução JUnit não foi fornecida. Após sua contribuição, executar mvn clean test novamente e atualizar os resultados antes de encerrar o aceite.

## Arquivos alterados e novos

Modificados:

- server/src/main/java/com/projetok/domain/movement/CollisionService.java.
- server/src/main/java/com/projetok/domain/movement/MovementService.java.
- server/src/main/java/com/projetok/domain/skill/LightningStepService.java.
- server/src/main/java/com/projetok/prototype/DebugWorld.java.
- server/src/main/java/com/projetok/prototype/DebugSnapshot.java.
- server/src/main/java/com/projetok/prototype/DebugPanel.java.
- server/src/test/java/com/projetok/domain/skill/LightningStepTestFixtures.java.
- README-PROTOTIPO.md e docs/HISTORICO-TECNICO.md.

Novos:

- server/src/main/java/com/projetok/domain/movement/CollisionCategory.java.

- server/src/main/java/com/projetok/domain/movement/CollisionPolicy.java.
- server/src/test/java/com/projetok/domain/movement/CollisionPolicyTest.java.
- server/src/test/java/com/projetok/domain/movement/CollisionRecoveryTest.java.
-
server/src/test/java/com/projetok/domain/skill/LightningStepSelectiveCollisionTest.java.
- server/src/test/java/com/projetok/prototype/SelectiveDashIntegrationTest.java.
- tools/gerar_pdf_task006_1.py.
- docs/TASK-006-1.md, TASK-006-1-testes.txt e TASK-006-1.diff.
- docs/Spirit_Gate_TASK-006-1.pdf e TASK-006-1-colisao-seletiva.png.

## Reprodução e limitações

Na pasta server, executar mvn clean test. Na pasta Projeto-K, python -B tools/gerar_pdf_task006_1.py regenera o PDF com a biblioteca padrão do Python. A captura é produzida em server/target pelo teste e copiada para docs na entrega.

Conferência jogando: abrir rodar-prototipo.cmd, usar H para hitboxes, atravessar um mob com Shift, mirar contra as paredes existentes, testar diagonais e bordas, observar término livre e volta do WASD; conferir ataque básico, Q/Freeze e R com Shift mantido. O cenário padrão não contém arbustos físicos.

O retorno é uma correção de posição no encerramento e pode produzir um recuo visível se o destino estiver ocupado. A rota cresce com os ticks de uma ativação; a arena usa 30 Hz. A busca de saída consulta os obstáculos e candidatos da rota; não foi introduzido cache de trajetórias ou otimização para milhares de atores. Em magnitudes extremas de float, o ajuste conservador de oito ULPs pode recusar uma solução matemática; validação final impede aceitar penetração.

O diff compara os fontes/documentos com o estado inspecionado no início desta tarefa. Artefatos PDF/PNG são entregues separadamente; server/target e dados locais não integram o diff. Não houve merge ou publicação. Aguardar sua implementação, conferência interativa e revisão técnica antes da TASK-007.